

Fundamentals of Web Applications

Frontend Development: Events and Interaction in JavaScript

Lecture 20 – May 20, 2026

Michael Soprano — michael.soprano@uniud.it

University of Udine — Department of Mathematics,
Computer Science, and Physics (DMIF)

Outline

1. Event-Driven Interaction
2. Registering Event Listeners
3. Reading Event Data
4. Interface State and Feedback
5. Form Validation and Controlled Submission

The Missing Trigger

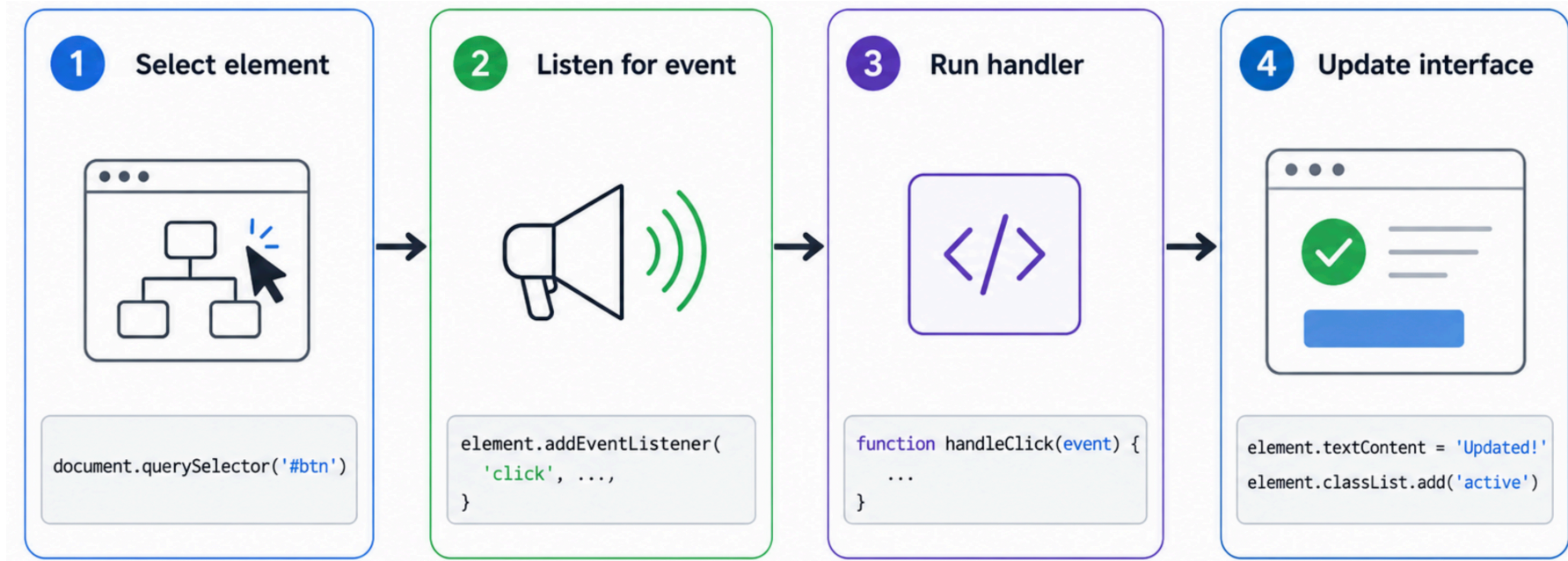
- DOM manipulation lets scripts **change the document**
- Interactive interfaces also need to know **when** to change it
- A user may **click, type**, select an option, or submit a form
- The browser reports these actions through **events**
- Events connect **user actions** to **JavaScript logic**

Events in the Browser

- An **event** is a signal that something happened
- Events can come from the **user**, the **document**, or the **browser window**
- A script can register code for a selected **event type**
- The registered code runs only when that event **occurs**
- The document can respond without being **reloaded**

Event Handling Pattern

- Event handling connects a user action to an **interface update**
- The code first selects an element and registers an **event listener**
- When the event occurs, the handler runs and updates the **DOM**



Event Categories

- Events describe different kinds of **browser activity**
- Some events come from **direct user interaction**
- Others come from forms, the keyboard, the document, or the window
- The event type tells JavaScript what kind of situation it should **react to**

Category	Examples	Usage
Pointer	<code>click</code> , <code>mousedown</code>	Actions and selections
Keyboard	<code>keydown</code> , <code>keyup</code>	Shortcuts and typed input
Form	<code>input</code> , <code>change</code> , <code>submit</code>	Feedback and validation
Document	<code>DOMContentLoaded</code>	Initialization
Window	<code>resize</code> , <code>scroll</code>	Viewport changes

Registering Event Listeners

- An event listener connects an **event type** to a **handler function**
- The browser stores the listener after the script runs
- The handler runs later, when the selected **event occurs**
- This pattern keeps **interaction logic** separate from the HTML structure

Listener Method: `addEventListener()`

- `addEventListener()` is called on the **element** to observe
- The first argument is the **event type**
- The second argument is the **handler function**
- The handler runs when the event **fires**

```
const button = document.querySelector(".save-button")

button.addEventListener("click", () => {
  console.log("Button clicked")
})
```

Named Handler Functions

- A handler can be written as a **named function**
- The function name is passed to `addEventListener()`
- The function is **not called** during registration
- The browser calls it later when the **event occurs**

```
const button = document.querySelector(".save-button")

function handleSaveClick() {
  console.log("Saving changes")
}

button.addEventListener("click", handleSaveClick)
```

Handler Reference

- `addEventListener()` expects a **function reference**
- Writing the function name passes the function itself
- Adding parentheses calls the function **immediately**
- The handler should run when the event occurs, not while the listener is registered

```
button.addEventListener("click", handleSaveClick) // correct
```

```
button.addEventListener("click", handleSaveClick()) // wrong
```

Event-Driven Execution

- The script first performs the **setup**
- The listener is stored by the browser
- The handler does not run while it is being **registered**
- It runs only when the selected **event fires**
- The same document can react to actions that happen later

```
button.addEventListener("click", () => {  
    statusMessage.textContent = "Click handled."  
})
```

Script-Based Behavior

- HTML should describe the **interface structure**
- JavaScript should describe the **interaction behavior**
- Event listeners **keep behavior in the script**

```
<button class="save-button">Save changes</button>
```

```
const button = document.querySelector(".save-button")  
button.addEventListener("click", handleSaveClick)
```

Managing Listeners

- More than one listener can be registered on the same **element**
- Separate listeners can handle different **concerns**
- A listener can be removed when it is no longer needed
- Removal requires the same **function reference**
- Named functions make listener management easier

```
function handleSaveClick() {  
    console.log("Saving changes")  
}  
  
function updateInterface() {  
    console.log("Updating interface")  
}  
  
button.addEventListener("click", handleSaveClick)  
button.addEventListener("click", updateInterface)  
  
button.removeEventListener("click", handleSaveClick)
```

Example: Source Code

- Sample **resource**: AG_event-listener-button/index.html
- The document contains a **button** and a **status message**
- The script registers a listener for the **button click**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Event Listener Button</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="page">
    <h1>Course Settings</h1>
    <button class="save-button">Save changes</button>
    <p class="status-message">No changes saved yet.</p>
  </main>
</body>
</html>
```

Example: Compact CSS

- Sample **resource**: [AG_event-listener-button/style.css](#)
- The message has a **neutral state** and a **visible saved state**
- JavaScript switches the state by adding a **CSS class**

```
body {  
  font-family: Arial, Helvetica, sans-serif;  
  line-height: 1.5;  
}  
  
.page {  
  max-width: 32rem;  
  margin: 2rem auto;  
}  
  
.status-message {  
  color: #555555;  
}  
  
.status-message.is-saved {  
  color: #1f4e79;  
  font-weight: 700;  
}
```

Example: Source Code

- Sample **resource**: `AG_event-listener-button/script.js`
- The handler updates the message only after the **click event**
- The DOM change is the response to the **interaction**

```
const button = document.querySelector(".save-button")
const statusMessage = document.querySelector(".status-message")

function handleSaveClick() {
  statusMessage.textContent = "Changes saved."
  statusMessage.classList.add("is-saved")
}

button.addEventListener("click", handleSaveClick)
```

Example: Result

- Before the click, the interface shows the **initial state**
- Clicking the button runs the registered **handler**
- After the click, the message **text** changes
- The CSS class changes the **visual state**

Before click

Course Settings

Save changes

No changes saved yet.

After click

Course Settings

Save changes

Changes saved.

Inside the Handler

- Registering a listener defines **when** code should run
- The handler often needs to know **what happened**
- The browser can pass an **event object** to the handler
- Event data connects the response to the concrete **user action**

```
button.addEventListener("click", event => {  
  console.log(event)  
})
```

Event Name: `type`

- `event.type` identifies the **event type** that fired
- The same handler can be reused for different **events**
- Reading the type helps inspect which interaction reached the handler
- This is useful when testing or debugging **event behavior**

```
function logEventType(event) {  
  console.log(`Handled event: ${event.type}`)  
}  
  
button.addEventListener("click", logEventType)  
textInput.addEventListener("input", logEventType)
```

Event Origin: `target`

- `event.target` identifies the element where the event **started**
- For a button click, it is usually the **clicked button**
- For text input, it is usually the **edited input field**
- The target connects the handler to the element involved in the **action**

```
button.addEventListener("click", event => {  
  const clickedElement = event.target  
  console.log(clickedElement.textContent)  
})
```

Listener Element: `currentTarget`

- `event.currentTarget` identifies the element where the listener is **registered**
- It refers to the element whose handler is currently **running**
- It is useful when the same handler is reused across several **elements**
- The handler can read the observed element without relying on an external variable

```
function logButton(event) {  
  const observedButton = event.currentTarget  
  console.log(observedButton.className)  
}  
  
saveButton.addEventListener("click", logButton)  
cancelButton.addEventListener("click", logButton)
```

Nested Event Flow

- `target` and `currentTarget` often refer to the same **element**
- They can differ when the event starts inside a nested **child element**
- The event starts from the inner element and reaches the element with the **listener**
- This prepares more advanced patterns based on event **propagation**

```
<button class="save-button">  
  <span>Save changes</span>  
</button>
```

```
button.addEventListener("click", event => {  
  console.log("Started from:", event.target)  
  console.log("Handled by:", event.currentTarget)  
})
```

Comparing Event Data

- Different event types expose different **information**
- Some events describe a changed **value**
- Other events describe a user **action**
- The handler reads the properties that match the intended **response**

```
input event  → current value  
keyup event  → released key
```

Field Value: `value`

- Form controls expose their current value through **DOM properties**
- During an `input` event, `event.target.value` reads the edited **field value**
- The value reflects the current content of the control
- Counters, previews, and validation hints often depend on this **updated value**

```
titleInput.addEventListener("input", event => {  
  const currentTitle = event.target.value  
  
  preview.textContent = currentTitle  
})
```

Keyboard Key: `key`

- Keyboard events describe a **keyboard interaction**
- `event.key` stores the pressed or released **key value**
- These events are not limited to forms or text fields
- A listener can observe a focused control or the whole **document**

```
document.addEventListener("keydown", event => {  
  if (event.key === "Escape") {  
    closePanel()  
  }  
})
```

Example: Source Code

- Sample **resource**: AH_reading-event-data/index.html
- The example observes the same **textarea** with two different events
- `input` updates the counter from the current **text value**
- `keyup` updates the message from the last **released key**

```
←!— Head omitted for brevity —→  
<body>  
  <main class="page">  
    <h1>Message Draft</h1>  
  
    <label for="message">Message</label>  
    <textarea id="message" class="message-input"></textarea>  
  
    <p class="counter">0 characters</p>  
    <p class="last-key">Last key: none</p>  
  </main>  
</body>  
</html>
```

Example: Source Code

- Sample **resource**: `AH_reading-event-data/script.js`
- The `input` handler reads `event.target.value`
- The `keyup` handler reads `event.key`
- The two handlers respond to related actions but use different **event data**

```
const messageInput = document.querySelector(".message-input")
const counter = document.querySelector(".counter")
const lastKey = document.querySelector(".last-key")

messageInput.addEventListener("input", event => {
  const characterCount = event.target.value.length
  counter.textContent = `${characterCount} characters`
})

messageInput.addEventListener("keyup", event => {
  lastKey.textContent = `Last key: ${event.key}`
})
```

Example: Result

- Before typing, the interface shows the **initial state**
- Typing in the textarea fires repeated **input events**
- Releasing a key fires a **keyup event**
- The two handlers update different pieces of **feedback**

Before typing

Message Draft

Message

0 characters

Last key: none

After typing

Message Draft

Message

Hello

5 characters

Last key: o

Visible State Changes

- Event handlers often update the **state** shown by the interface
- A state can be represented by **text**, **attributes**, or **CSS classes**
- Feedback tells the user that an action has been received
- Good interaction keeps the visible state **clear**, **predictable**, and **consistent**

Coordinated Feedback

- A single interaction may affect several **interface elements**
- The panel visibility, button label, and status message should stay **aligned**
- The handler can compute the new state once
- Text and classes can then be updated from that same **state**

```
const isVisible = detailsPanel.classList.toggle("is-visible")

toggleButton.textContent = isVisible ? "Hide details" : "Show details"

statusMessage.textContent = isVisible
  ? "Details are visible."
  : "Details are hidden."

statusMessage.classList.toggle("is-active", isVisible)
```

Example: Source Code

- Sample **resource**: AI_interface-state-feedback/index.html
- The example uses a button to show or hide **extra details**
- The handler updates the panel, the button label, and the **status message**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Interface State and Feedback</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="page">
    <h1>Course Topic</h1>

    <button class="toggle-button">Show details</button>
    <p class="status-message">Details are hidden.</p>

    <section class="details-panel">
      <h2>Event Handling</h2>
      <p>Events connect user actions to JavaScript logic.</p>
    </section>
  </main>
</body>
</html>
```

Example: Compact CSS

- Sample **resource**: `AI_interface-state-feedback/style.css`
- The panel is hidden by **default**
- The `is-visible` class shows the panel
- The `is-active` class emphasizes the current **status**

```
body { font-family: Arial, Helvetica, sans-serif; line-height: 1.5; }
.page { max-width: 32rem; margin: 2rem auto; }

.details-panel {
  display: none;
  margin-top: 1rem;
  padding: 1rem;
  border: 1px solid #cccccc;
}

.details-panel.is-visible { display: block; }

.status-message.is-active {
  color: #1f4e79;
  font-weight: 700;
}
```

Example: Source Code

- Sample **resource**: [AI_interface-state-feedback/script.js](#)
- The handler computes the new **visibility state**
- Text and classes are updated from the same **state**
- The interface remains consistent after each **click**

```
const toggleButton = document.querySelector(".toggle-button")
const statusMessage = document.querySelector(".status-message")
const detailsPanel = document.querySelector(".details-panel")

toggleButton.addEventListener("click", () => {
  const isVisible = detailsPanel.classList.toggle("is-visible")

  toggleButton.textContent = isVisible ? "Hide details" : "Show details"

  statusMessage.textContent = isVisible
    ? "Details are visible."
    : "Details are hidden."

  statusMessage.classList.toggle("is-active", isVisible)
})
```

Example: Result

- Before the click, the details panel is **hidden**
- After the click, the panel becomes **visible**
- The button label changes to match the next available **action**
- The status message describes the current **state**

Before click

Course Topic

Show details

Details are hidden.

After click

Course Topic

Hide details

Details are visible.

Event Handling

Events connect user actions to JavaScript logic.

Before Submission

- Forms collect values that often need to be **checked** before submission
- JavaScript can provide immediate **feedback** while the user interacts
- A submit handler can decide whether the form is ready to be **processed**
- Client-side validation improves the interface, but the backend still enforces the **rules**

Form Events

- Forms produce events while the user **edits**, **changes**, and **submits** data
- `input` fires while a text value is being edited
- `change` fires when a committed value changes
- `submit` fires when the form is about to be submitted
- These events let JavaScript check values before data leaves the **interface**

Event	Typical role
<code>input</code>	Live feedback while text changes
<code>change</code>	Updates after selections or checkbox changes
<code>submit</code>	Final validation before submission

Live Feedback: `input`

- The `input` event fires whenever the current **value changes**
- Text fields and textareas can update feedback while the user types
- The handler can read the current value through `event.target.value`
- Counters, previews, and validation hints often follow this pattern

```
emailInput.addEventListener("input", event => {  
  const email = event.target.value  
  
  feedback.textContent = `${email.length} characters`  
})
```

Committed Choices: `change`

- The `change` event fires when the user commits a new **value**
- Select menus, checkboxes, and radio buttons often use this event
- The handler can read state such as `checked` or `value`
- Related controls can be updated after the choice changes

```
termsCheckbox.addEventListener("change", event => {  
  submitButton.disabled = !event.target.checked  
})
```

Field Rules

- JavaScript can check whether form values satisfy **interface rules**
- A rule can inspect text length, selected options, or checkbox state
- The result can be stored as a simple **valid** or **invalid** state
- Validation logic should stay readable and close to the form behavior

```
function isEmailComplete() {  
    return emailInput.value.trim().length > 0  
}  
  
function isMaterialSelected() {  
    return materialSelect.value.length > 0  
}  
  
function isRequestConfirmed() {  
    return confirmCheckbox.checked  
}
```

Validation Feedback

- Validation should produce clear **feedback**
- Error messages should be close to the related **form control**
- `textContent` is appropriate for plain text messages
- CSS classes can mark controls and messages as **invalid**

```
function showError(messageElement, message) {  
  messageElement.textContent = message  
  messageElement.classList.toggle("is-visible", message.length > 0)  
}
```

Disabled Controls: `disabled`

- The `disabled` property controls whether a form control can be used
- A disabled submit button cannot be **clicked**
- Event handlers can enable it when required conditions are met
- CSS can style disabled controls with `:disabled`

```
submitButton.disabled = true

confirmCheckbox.addEventListener("change", event => {
  submitButton.disabled = !event.target.checked
})
```

Form Submission: `submit`

- The `submit` event belongs to the **form**
- It fires when the user submits through a button or the keyboard
- A submit handler can inspect all current **form values**
- The handler is the right place for a final **validation check**

```
form.addEventListener("submit", event => {  
  console.log("Submitting form")  
})
```

Default Browser Behavior

- Some elements have built-in **browser behavior**
- A form submission usually sends data and may reload or navigate
- A link click usually navigates to its **destination**
- JavaScript can intercept these actions when the interface needs to stay on the page

```
form.addEventListener("submit", event => {  
  event.preventDefault()  
})
```

Controlled Submission: `preventDefault()`

- `preventDefault()` cancels the built-in action for the current event
- In a submit handler, it keeps the document from **navigating away**
- The script can validate the fields and update feedback
- If the values are valid, the interface can prepare the next **application step**

```
form.addEventListener("submit", event => {  
  event.preventDefault()  
  
  if (!validateForm()) {  
    feedbackMessage.textContent = "Check the highlighted fields."  
    return  
  }  
  
  feedbackMessage.textContent = "The request is ready."  
})
```

Submitted Values: `FormData`

- A form submission is based on the **named controls** inside the form
- `FormData` reads the values that would be submitted
- Each value is accessed through the control **name**
- A checked checkbox contributes its configured **value**

```
const formData = new FormData(form)

const email = formData.get("email")
const material = formData.get("material")
const confirmed = formData.get("confirmed")

console.log(email)
console.log(material)
console.log(confirmed)
```

Validation Boundaries

- **Client-side validation** gives immediate feedback in the browser
- It improves the interaction before data is sent
- It cannot be trusted as the only validation layer
- **Server-side validation** is still necessary for data that reaches the backend
- The frontend helps the user, while the backend enforces the rules

Example: Form Structure

- Sample **resource**: AJ_form-js-validation/index.html
- The form disables native validation with `novalidate`
- Each control has a **name** for later `FormData` extraction
- Feedback paragraphs are placed close to their related **controls**

```
<form class="materials-form" novalidate>
  <label for="email">Email</label>
  <input id="email" class="email-input" name="email" type="email">
  <p class="field-feedback email-feedback"></p>

  <label for="material">Material</label>
  <select id="material" class="material-select" name="material">
    <option value="">Choose a material</option>
    <option>JavaScript event examples</option>
    <option>DOM manipulation exercises</option>
  </select>
  <p class="field-feedback material-feedback"></p>
```

←!— Confirmation and submit controls follow —→

Example: Confirmation Control

- Sample **resource**: AJ_form-js-validation/index.html
- The checkbox stores a named **confirmation value**
- The submit button starts as **disabled**
- JavaScript will enable it only when the form is **valid**

```
<label>
  <input class="confirm-checkbox" name="confirmed" type="checkbox" value="yes">
  I confirm this materials request
</label>
<p class="field-feedback confirm-feedback"></p>

<button class="submit-button" type="submit" disabled>
  Prepare request
</button>
</form>
```

Example: Summary Markup

- Sample **resource**: AJ_form-js-validation/index.html
- The summary starts **hidden**
- Empty spans mark the places filled by JavaScript
- The script inserts submitted values as **plain text**

```
<section class="request-summary" hidden>
  <h2>Request prepared</h2>
  <p><strong>Material:</strong> <span class="summary-material"></span></p>
  <p><strong>Recipient:</strong> <span class="summary-recipient"></span></p>
</section>

<p class="feedback-message">
  Complete the form to prepare the request.
</p>
```

Example: Compact CSS

- Sample **resource**: [AJ_form-js-validation/style.css](#)
- CSS hides feedback messages by **default**
- JavaScript changes visibility and state through **properties** and **classes**

```
label {  
  display: block;  
  margin-top: 1rem;  
}  
  
input, select, button { font: inherit; }  
.field-feedback.is-visible { display: block; }  
.submit-button:disabled { cursor: not-allowed; opacity: 0.55; }  
  
.field-feedback {  
  display: none;  
  margin: 0.25rem 0 0;  
  color: #8a1f11;  
}  
  
.request-summary.is-ready {  
  margin-top: 1rem;  
  padding: 1rem;  
  border-left: 0.35rem solid #1f4e79;  
}
```

Example: Form Controls

- Sample **resource**: [AJ_form-js-validation/script.js](#)
- The script selects the form and its **controls**
- These elements provide the current values used by **validation**

```
const form = document.querySelector(".materials-form")

const emailInput = document.querySelector(".email-input")
const materialSelect = document.querySelector(".material-select")
const confirmCheckbox = document.querySelector(".confirm-checkbox")
const submitButton = document.querySelector(".submit-button")
```

Example: Feedback Elements

- Sample **resource**: `AJ_form-js-validation/script.js`
- The script also selects the feedback and summary **targets**
- These elements are updated after validation and **submission**

```
const emailFeedback = document.querySelector(".email-feedback")
const materialFeedback = document.querySelector(".material-feedback")
const confirmFeedback = document.querySelector(".confirm-feedback")
const feedbackMessage = document.querySelector(".feedback-message")

const requestSummary = document.querySelector(".request-summary")
const summaryMaterial = document.querySelector(".summary-material")
const summaryRecipient = document.querySelector(".summary-recipient")
```

Example: Feedback Helper

- Sample **resource**: `AJ_form-js-validation/script.js`
- A helper keeps feedback updates **consistent**
- Empty messages hide the error state
- Non-empty messages mark the feedback as **visible**

```
function showError(messageElement, message) {  
  messageElement.textContent = message  
  messageElement.classList.toggle("is-visible", message.length > 0)  
}
```

Example: Validation Logic

- Sample **resource**: [AJ_form-js-validation/script.js](#)
- The validation function checks all required **form values**
- The email rule checks a readable **structure**, not full legal validity
- Each invalid field receives a specific **feedback message**

```
function validateForm() {  
  const email = emailInput.value.trim()  
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/  
  
  const hasValidEmail = emailPattern.test(email)  
  const hasMaterial = materialSelect.value.length > 0  
  const isConfirmed = confirmCheckbox.checked  
  
  showError(emailFeedback, hasValidEmail ? "" : "Enter a valid email address.")  
  showError(materialFeedback, hasMaterial ? "" : "Choose a material.")  
  showError(confirmFeedback, isConfirmed ? "" : "Confirm the request.")  
  
  return hasValidEmail && hasMaterial && isConfirmed  
}
```

Example: State Updates

- Sample **resource**: [AJ_form-js-validation/script.js](#)
- `input` and `change` events update the same **form state**
- The button is enabled only when the form passes **validation**
- The global feedback describes the current **readiness**

```
function updateRequestState() {  
  const isValid = validateForm()  
  
  submitButton.disabled = !isValid  
  
  feedbackMessage.textContent = isValid  
    ? "The request is ready to be prepared."  
    : "Complete the form to prepare the request."  
}  
  
emailInput.addEventListener("input", updateRequestState)  
materialSelect.addEventListener("change", updateRequestState)  
confirmCheckbox.addEventListener("change", updateRequestState)
```

Example: Submit Handling

- Sample **resource**: `AJ_form-js-validation/script.js`
- `preventDefault()` keeps the document on the same page
- The submit handler performs a final **validation check**
- `FormData` reads the values prepared for **submission**

```
form.addEventListener("submit", event => {
  event.preventDefault()

  if (!validateForm()) {
    feedbackMessage.textContent = "Check the highlighted fields."
    return
  }
  const formData = new FormData(form)
  summaryMaterial.textContent = formData.get("material")
  summaryRecipient.textContent = formData.get("email")
  requestSummary.hidden = false
  requestSummary.classList.add("is-ready")

  feedbackMessage.textContent = "The request preview is ready."
})
```

Example: Initial State

- The form starts with missing **requirements**
- Feedback messages explain what needs to be completed
- The submit button is still **disabled**
- No request summary is visible yet

Request Course Materials

Email

Enter a valid email address.

Material

Choose a material

Choose a material.

Prepare request

Validation feedback explains why the form is not ready yet

Example: Submitted State

- After valid input, the button becomes **available**
- The submit handler keeps the document on the same page
- `FormData` provides the submitted **values**
- JavaScript fills the visible **request summary**

Request Course Materials

Request prepared

Material: JavaScript event examples

Recipient: student@example.com

The request preview is ready.

Summary

- Events connect **user actions** to JavaScript logic
- `addEventListener()` registers handlers that run **later**
- The event object describes what happened and where it **started**
- Handlers use DOM updates to keep interface state **coherent**
- Form events support validation, feedback, and **controlled submission**

Advanced Event Patterns

- The core workflow covers the most common **interaction structure**
- Larger interfaces introduce more advanced **event behavior**
- Events can travel through nested elements before and after reaching the target
- Some interactions can be handled from a **shared container**
- Frequent events may require patterns that keep the interface responsive

Bibliography

- Duckett, J. (2014). *JavaScript & jQuery: Interactive Front-End Web Development*. John Wiley & Sons
 - Chapter 6: Events
- MDN Web Docs. *Event reference*
 - <https://developer.mozilla.org/en-US/docs/Web/Events>
- MDN Web Docs. *EventTarget: addEventListener()*
 - <https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener>

Bibliography (cont.)

- MDN Web Docs. *Event*
 - <https://developer.mozilla.org/en-US/docs/Web/API/Event>
- MDN Web Docs. *HTMLElement: input event*
 - https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/input_event
- MDN Web Docs. *HTMLElement: change event*
 - https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/change_event
- MDN Web Docs. *HTMLFormElement: submit event*
 - https://developer.mozilla.org/en-US/docs/Web/API/HTMLFormElement/submit_event

Bibliography (cont.)

- MDN Web Docs. *Event: preventDefault()*
 - <https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault>
- MDN Web Docs. *HTML attribute: disabled*
 - <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/disabled>
- MDN Web Docs. *Client-side form validation*
 - https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation
- MDN Web Docs. *FormData*
 - <https://developer.mozilla.org/en-US/docs/Web/API/FormData>